Cómo pasar el proyecto entre dispositivos

No va a dar errores. La forma más limpia es:

1. Comprime la carpeta del proyecto en ZIP (todo lo que hay en pro01-Nginx-Python-Flask-FastAPI-MySQL)
2. En clase, borra la carpeta vieja del proyecto y extrae el ZIP nuevo en su lugar
3. Reconstruye los contenedores desde cero (esto aplica el nuevo init.sql con todos los cambios):
   docker-compose down -v
   docker-compose up --build -d
   El -v borra los volúmenes antiguos (datos de BD, etc.) y --build reconstruye la imagen del backend con el nuevo código. Al arrancar, MySQL ejecutará el init.sql nuevo con las tablas, el trigger, el SP y los datos seed.
   Si no quieres perder las reservas que hayas hecho en clase, en vez de down -v puedes hacer:
   docker-compose down
   docker-compose up --build
   Luego aplicar solo los cambios de estructura (tabla auditoria, trigger, SP) con:
   docker exec -i db-mysql mysql -uroot -proot bd1 < db\update.sql
   Pero lo más seguro y limpio para una presentación es reconstruir desde cero con down -v así partes con datos frescos y todo funcionando al 100%.

Esto es en caso de que den igual las reservas que se se hayan hecho en los dispostivos.

```
# Abre PowerShell o CMD

# 1. Borra la carpeta vieja del proyecto (si existe)
rm -r C:\Users\...\pro01-Nginx-Python-Flask-FastAPI-MySQL

# 2. Extrae el ZIP en la misma ruta

# 3. Entra al proyecto
cd C:\Users\...\pro01-Nginx-Python-Flask-FastAPI-MySQL

# 4. Elimina contenedores y volúmenes viejos
docker-compose down -v

# 5. Construye y arranca todo desde cero
docker-compose up --build -d

# 6. Espera 20 segundos a que todo arranque

# 7. Abre en el navegador: http://localhost

El down -v borra los datos antiguos de la BD. Al hacer up --build, MySQL ejecuta el
nuevo init.sql con todas las tablas, el trigger y el stored procedure. El backend y
frontend se actualizan también.

No necesitas hacer nada más. El proyecto queda funcionando igual que en tu casa.
```